In [1]:
```
# Install once if needed:
# pip install pandas gensim

# 1. Import libraries
import re
import pandas as pd
from gensim.models import Word2Vec
```

In [3]:
```
# 2. Load Twitter dataset
# Keep the CSV in the same folder as your notebook.
df = pd.read_csv("twitter_dataset.csv")

# Change "text" below if your dataset uses a different tweet column name.
print(df.columns)
```

```
Index(['Tweet_ID', 'Username', 'Text', 'Retweets', 'Likes', 'Timestamp'], dtype='object')
```

In [4]:
```
# 3. Clean and tokenize tweets
def clean_tweet(tweet):
    tweet = str(tweet).lower()
    tweet = re.sub(r"http\S+|www\S+", "", tweet)
    tweet = re.sub(r"@\w+", "", tweet)
    tweet = re.sub(r"#", "", tweet)
    tweet = re.sub(r"[^a-z\s]", " ", tweet)
    return tweet.split()

sentences = df["Text"].dropna().apply(clean_tweet).tolist()
```

In [5]:
```
# 4. Train CBOW Word2Vec model (sg=0 means CBOW)
cbow_model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=2,
    sg=0,
    workers=4,
    epochs=5
)
```

```
# 5. Display vocabulary size
print("Vocabulary Size:", len(cbow_model.wv))
```

```
Vocabulary Size: 971
```

In [6]:
```
# 6. Check a word embedding
word = "good"

if word in cbow_model.wv:
    print("Embedding for", word, ":")
    print(cbow_model.wv[word])

    print("Similar words:")
    print(cbow_model.wv.most_similar(word, topn=5))
else:
    print("Word not found in vocabulary:", word)
```

```
Embedding for good :
[-0.15985087  0.21823342  0.13071911  0.04112892  0.06540854 -0.2749112
 -0.0924641   0.29516357 -0.01965681  0.07634516 -0.3171494  -0.27207252
 -0.0677553   0.06031457  0.05673011  0.00849167 -0.01059643 -0.07323175
  0.08533946 -0.17002651  0.12409209  0.07448993  0.01680476  0.01379286
 -0.08809018  0.07592494 -0.08170202 -0.18165663  0.06004628 -0.05201999
  0.20434181  0.0093725  -0.14771403  0.19363405 -0.07606583  0.04945625
 -0.11707823 -0.26374048 -0.20975494 -0.26215887 -0.00238835  0.02942238
  0.12510206 -0.07242528 -0.07708246 -0.03168747 -0.128293    0.10151973
  0.04010897  0.14122666  0.01636333 -0.1786627  -0.13183348  0.07093862
 -0.3143555   0.15557991  0.17676647  0.0158455  -0.22356905 -0.0771941
  0.11943891  0.21528989 -0.2831058  -0.06679555 -0.19387859  0.0102567
  0.00060241 -0.00905657 -0.03711433  0.08017078 -0.08864509 -0.11401182
  0.21171637 -0.1110809   0.09329972  0.07784547 -0.08686964 -0.09012338
 -0.20356417  0.11413982 -0.09035967 -0.09112421 -0.00866135  0.25665146
  0.18056947  0.06486697 -0.25840393  0.30401233  0.13785188  0.16381484
  0.06238132 -0.054793   -0.09438036 -0.06496763  0.19119476  0.19001605
  0.18059577 -0.33510277  0.19957902 -0.08105937]
Similar words:
[('television', 0.9977869391441345), ('allow', 0.997784435749054), ('century', 0.9977635145187378), ('center', 0.9977622032165527), ('training', 0.9977262020111084)]
```

In [7]:
```
# 7. Check another word
word = "happy"
```

```
if word in cbow_model.wv:
    print("Similar words to", word, ":")
    print(cbow_model.wv.most_similar(word, topn=5))
else:
    print("Word not found in vocabulary:", word)
```

```
Similar words to happy :
[('bank', 0.9981306791305542), ('activity', 0.9981148838996887), ('rate', 0.9981127977371216), ('size', 0.998081624507904), ('a
ction', 0.9980769753456116)]
```

In [ ]:

In [ ]: